In [1]:

```
# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np

from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
```

In [2]:

```
# ============================================================
# 2. LOAD DATA
# ============================================================

tracks = pd.read_csv(
    "../data/raw/metadata/tracks.csv",
    header=[0, 1],
    index_col=0
)

features = pd.read_csv(
    "../data/raw/metadata/features.csv",
    header=[0, 1, 2],
    index_col=0
)

echonest = pd.read_csv(
    "../data/raw/metadata/echonest.csv",
    header=[0, 1, 2],
    index_col=0
)
```

In [3]:

```
# ============================================================
# 3. FILTER TO SMALL SUBSET
# ============================================================

tracks_small = tracks[tracks[("set", "subset")] == "small"]

labels = tracks_small[("track", "genre_top")]
splits = tracks_small[("set", "split")]
```

In [4]:

```
# ============================================================
# 4. FIND TRACKS THAT EXIST IN BOTH DATASETS
# ============================================================

common_tracks = tracks_small.index.intersection(echonest.index)

tracks_common = tracks_small.loc[common_tracks]

labels_common = tracks_common[("track", "genre_top")]
splits_common = tracks_common[("set", "split")]

features_common = features.loc[common_tracks]
echonest_common = echonest.loc[common_tracks]

print("Tracks with both features:", len(common_tracks))
```

```
Tracks with both features: 1294
```

In [5]:

```
# ============================================================
# 5. REMOVE MISSING LABELS AND KEEP NUMERIC FEATURES ONLY
# ============================================================
# Some Echonest columns contain text/date values.
# Machine learning models like SVM require numeric input only.
# Therefore, we keep only numeric columns from both datasets.

valid = labels_common.notna()

X_feat = features_common[valid]
X_echo = echonest_common[valid]

y = labels_common[valid]
split = splits_common[valid]

# Keep numeric columns only
X_feat = X_feat.select_dtypes(include=["number"])
X_echo = X_echo.select_dtypes(include=["number"])

# Replace infinite values with NaN
X_feat = X_feat.replace([np.inf, -np.inf], np.nan)
X_echo = X_echo.replace([np.inf, -np.inf], np.nan)

# Fill missing numeric values using column means
X_feat = X_feat.fillna(X_feat.mean())
X_echo = X_echo.fillna(X_echo.mean())

print("Numeric Features shape:", X_feat.shape)
print("Numeric Echonest shape:", X_echo.shape)
```

```
Numeric Features shape: (1294, 518)
Numeric Echonest shape: (1294, 244)
```

In [6]:

```
# ============================================================
# 6. COMBINE FEATURES + ECHONEST
# ============================================================
# After keeping numeric columns only, we combine both datasets.
# This allows us to test whether Echonest improves prediction.

X_combined = pd.concat([X_feat, X_echo], axis=1)

# Some column names may overlap, so we convert them to simple unique names.
X_feat.columns = [f"feat_{i}" for i in range(X_feat.shape[1])]
X_echo.columns = [f"echo_{i}" for i in range(X_echo.shape[1])]
X_combined.columns = [f"combined_{i}" for i in range(X_combined.shape[1])]

print("Features only:", X_feat.shape)
print("Echonest only:", X_echo.shape)
print("Combined:", X_combined.shape)
```

```
Features only: (1294, 518)
Echonest only: (1294, 244)
Combined: (1294, 762)
```

In [7]:

```
# ============================================================
# 7. CREATE TRAIN / VALIDATION / TEST SETS
# ============================================================

def split_data(X):
    return (
        X[split == "training"],
        X[split == "validation"],
        X[split == "test"]
    )

Xf_train, Xf_val, Xf_test = split_data(X_feat)
Xe_train, Xe_val, Xe_test = split_data(X_echo)
Xc_train, Xc_val, Xc_test = split_data(X_combined)

y_train = y[split == "training"]
y_val = y[split == "validation"]
y_test = y[split == "test"]
```

In [8]:

```
# ============================================================
# 8. SCALE DATA (IMPORTANT FOR SVM)
# ============================================================

scaler_f = StandardScaler()
scaler_e = StandardScaler()
scaler_c = StandardScaler()

Xf_train_s = scaler_f.fit_transform(Xf_train)
Xf_val_s = scaler_f.transform(Xf_val)
```

```
Xe_train_s = scaler_e.fit_transform(Xe_train)
Xe_val_s = scaler_e.transform(Xe_val)

Xc_train_s = scaler_c.fit_transform(Xc_train)
Xc_val_s = scaler_c.transform(Xc_val)
```

In [9]:

```
# ============================================================
# 9. TRAIN SVM MODELS
# ============================================================

svm = SVC(C=10, kernel="rbf")

# Features only
svm.fit(Xf_train_s, y_train)
val_pred_feat = svm.predict(Xf_val_s)

# Echonest only
svm.fit(Xe_train_s, y_train)
val_pred_echo = svm.predict(Xe_val_s)

# Combined
svm.fit(Xc_train_s, y_train)
val_pred_comb = svm.predict(Xc_val_s)
```

In [10]:

```
# ============================================================
# 10. COMPARE MODEL PERFORMANCE
# ============================================================

results = pd.DataFrame({
    "Model": [
        "Features Only",
        "Echonest Only",
        "Combined Features"
    ],
    "Validation Accuracy": [
        accuracy_score(y_val, val_pred_feat),
        accuracy_score(y_val, val_pred_echo),
        accuracy_score(y_val, val_pred_comb)
    ]
})

print("\nECHONEST COMPARISON RESULTS:")
display(results)
```

```
ECHONEST COMPARISON RESULTS:
```

	Model	Validation Accuracy
0	Features Only	0.627119
1	Echonest Only	0.576271
2	Combined Features	0.635593